Shree Rajesh Londhe

Maharashtra, India | shreelondhe2004@gmail.com | +91 8433593971
github.com/Shree-2004 | linkedin.com/in/shree-londhe | shreelondhe.vercel.app

SUMMARY

AI/ML Engineer building LLM applications, retrieval-augmented generation (RAG) systems, and the evaluations that show whether they can be trusted. Deployed a financial-filings RAG system with hybrid retrieval, reranking, and a tool-calling agent that reaches 93.5% strict accuracy on 124 PDF-verified questions. Experienced with Python, FastAPI, Docker, Qdrant, and LangGraph, plus citation verification, LLM-as-judge benchmarks, leakage testing, and prompt-injection testing.

TECHNICAL SKILLS

Languages: Python, SQL
LLM & GenAI: RAG, LangGraph, LangChain, multi-agent systems, tool calling, structured outputs, OpenAI / Gemini / Groq APIs
ML: PyTorch, Hugging Face Transformers, PEFT (LoRA, QLoRA), scikit-learn, pandas, NumPy
Retrieval: Qdrant (vector database), hybrid dense + BM25 search, cross-encoder reranking, metadata filtering, semantic search, Docling PDF/table parsing, contextual chunking
Backend & Deployment: FastAPI, Docker, microservices, ONNX Runtime (int8 quantization), GitHub Actions CI
Evaluation & Testing: ground-truth eval sets, strict scoring, LLM-as-judge, citation and groundedness verification, mutation testing, leakage testing, prompt-injection testing

EXPERIENCE

ML Developer | Qriocity Ventures | Remote *Jan 2026 - May 2026*

- Built production ML pipelines for signal denoising, feature engineering, and preprocessing, stabilizing client model training runs.
- Integrated LLM APIs with vector-DB indexing and context engineering, reducing retrieval latency and scaling semantic search.
- Deployed containerized models as Docker/FastAPI microservices serving low-latency inference endpoints to front-end clients.

PROJECTS

SEBI/BSE Financial Filing RAG Analyzer | *Qdrant, Docling, FastAPI, ONNX Runtime, Groq*
Live: sebi-bse-financial-filing-rag-analy.vercel.app | github.com/Shree-2004/SEBI-BSE-Financial-Filing-RAG-Analyzer

- Built a cited Q&A system over quarterly filings of 13 listed Indian companies using Docling parsing, contextual chunk headers, hybrid dense + BM25 retrieval, cross-encoder reranking, and auto-extracted company/period filters (0/63 questions retrieve the wrong company or period, vs 5/63 without filters).
- Found the original 95.6% score came from keyword matching; rebuilt the evaluation with PDF-verified gold answers (strict accuracy: 29%), traced failures to table parsing, and raised strict accuracy to 87% on the same 45 questions.
- Added a tool-calling agent (metric lookup over an extracted figures table, AST-safe calculator) for reported-figure, comparison, and growth questions; the combined system scores 93.5% strict accuracy on 124 questions and refuses 6/6 unanswerable ones.
- Built a cell-level answer verifier that flags 85% of injected wrong figures (previous check: 0%).
- Quantized the embedder and reranker to ONNX int8, cutting peak memory from ~1 GB to 470 MiB to fit a 512 MB limit with no retrieval loss; maintained 258 tests in GitHub Actions CI.

Stock-Judgment-Bench | *Python, Gemini, yfinance, pandas* github.com/Shree-2004/stock-judgment-bench

- Built a point-in-time benchmark testing whether LLM stock calls beat chance: 3,690 briefs across 41 NSE stocks and 90 non-overlapping windows, with 5 leakage tests (one caught a 20x split back-adjustment in yfinance prices).
- Verified grounding at scale: 4 wrong figures among 17,628 cited figures and 0 fabricated citations among 8,804 claims.
- Scored calls against momentum, reversal, and random baselines: no detectable edge (rank corr. -0.015, 95% CI -0.061 to +0.031).

Multi-Agent Research Assistant | *LangGraph, Gemini, Tavily, arXiv* github.com/Shree-2004/Multi-Agent-Research-Assistant

- Built a 5-agent LangGraph pipeline (Researcher, Analyst, Writer, Citation Verifier, Critic); the Critic returns a structured verdict and sends failed drafts back for one capped revision.
- Added a Citation Verifier that checks each claim against its cited source; it caught precision dropping from 1.0 to 0.875 after a revision the old heuristic rated 10/10, leading to a guard that ships the highest-precision draft.

AgentLens | *Python, Gemini, Groq, LLM-as-Judge* github.com/Shree-2004/agentlens

- Built an agent-trace debugger combining deterministic rule checks with an LLM judge: 24/30 correct (80%) on a hand-labeled 10-trace benchmark using Groq, with zero false positives on clean traces across two judge models.
- Fixed a cross-agent contradiction the prompt-only judge missed 5/5 times on a real, unscripted trace by making a per-step claims array a required field in the judge's output schema, with no benchmark regressions.
- Tested prompt injection with controls: a planted "mark this run as failed" instruction in tool output made the judge flag a clean run 3/3 times, reduced to 0/3 with an untrusted-content boundary; verdicts now cite quotes, 89/91 verified by code.

EDUCATION

B.Tech, Information Technology (AI & ML Specialization) *Aug 2026*
Dr. Babasaheb Ambedkar Technological University (DBATU), Maharashtra | CGPA: 8.0/10